

점검 교량 출장 계획 수립 자동화: 최적 경로 탐색 서비스 개발

Automation of On-Site Inspection Trip Planning for Bridges:
Development of an Optimal Route-Finding Service

2026.01.29

구조부 유지관리팀

정재석 학습사원

CONTENTS

- 01 프로젝트 개요
- 02 프로그램 사용 방법
- 03 최적 경로 탐색 방법
- 04 향후 발전 계획

프로젝트 개요(1/2)

프로그램 개발 목적 : 일정 계획 과정의 **자동화**

다수의 교량 점검 시
사람이 직접 계획



최적 경로와 일정 산출하는
프로그램 필요

일정 수립 과정

점검 교량
리스트



최적 경로
계산



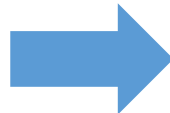
결과 도출



"사람이 직접 계산"

As-Is

주관적 동선, 번거로움



"최적 경로 산출 자동화"

To-Be

신속, 정확성 확보



프로젝트 개요(2/2)

최적 경로 탐색 서비스 프로그램의 기능적 요구사항

기능적 요구사항

점검 교량
리스트

입력단계

- 점검 교량 사용자 직접 구성
- 교량 정보를 포함한 데이터 베이스 구성
- 점검 교량 위치 지도 표시
- 출발 위치 지정

최적 경로
계산

계산 단계

- 카카오 API를 사용
- 경로 최적화 알고리즘 구현
(단, 하루 근무시간 8시간제한)

결과 도출

출력 단계

- 일자별 방문경로 시각화
- 일자별 레이어 구분
- 방문 순서 마커 및 경로 표시

프로그램 개발 방향성 (사용자 친화적)

“별도의 설치 없이 배포용 폴더를 받으면 실행 가능하도록 구성”

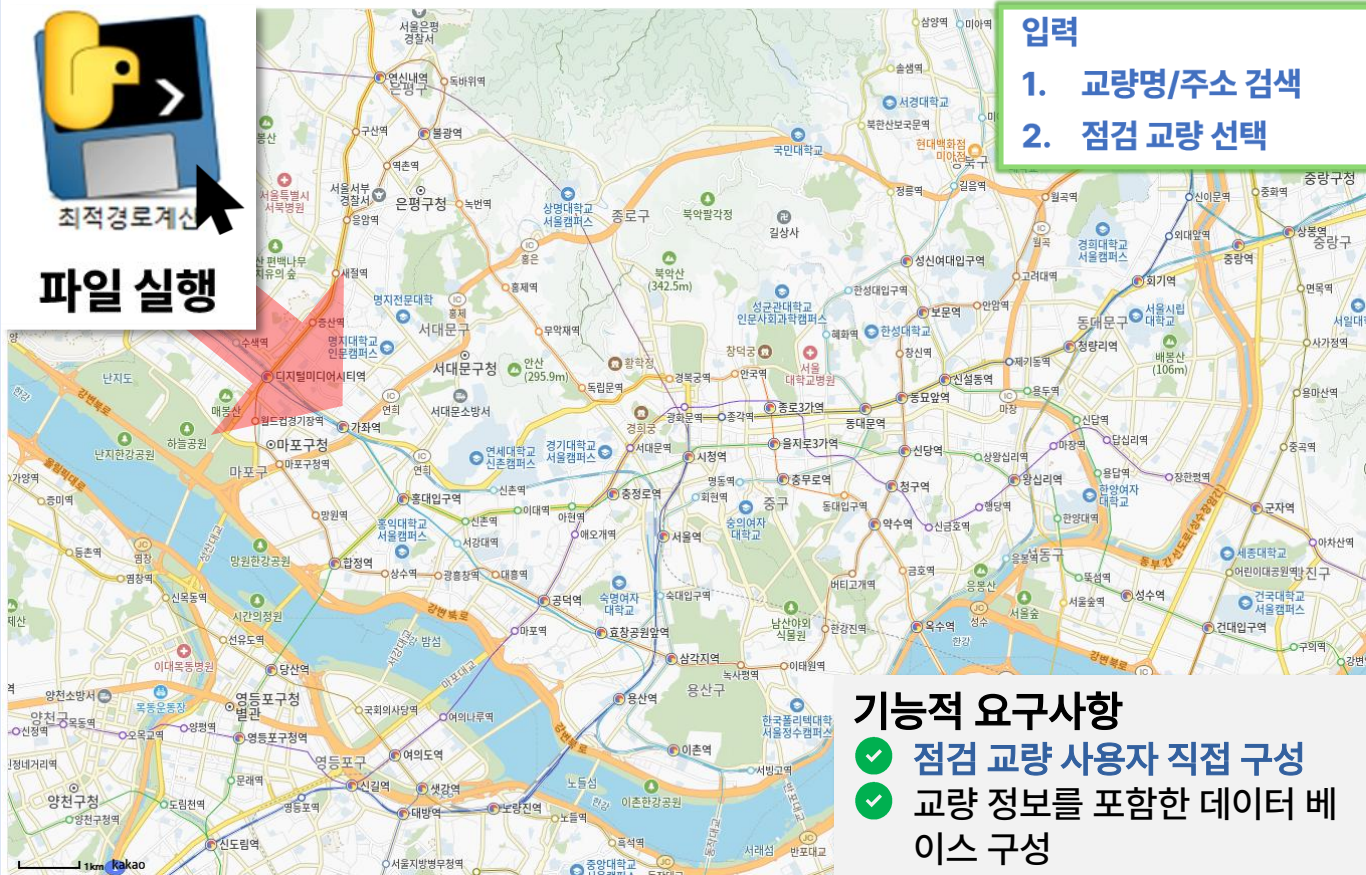
프로그램 사용 방법(1/3)

실행 시 첫 화면: 점검 교량 선택

아파가다리



파일 실행



기능적 요구사항

- ✓ 점검 교량 사용자 직접 구성
- ✓ 교량 정보를 포함한 데이터 베이스 구성

게시 ON: route_cache를 읽고/쓰며, 있으면 재사용합니다.

점검 대상 선택(DB)

관할: (전체)

검색: 교량명/주소 검색

전체선택

선택해제

선택완료

선택 0개

- ☐ **가송1교** (확장부) (대전청)
천안시 동남구 풍세면 풍서리 149-15
- ☐ **가송2교** (대전청)
천안시 동남구 풍세면 가송리 산 41-3
- ☐ **갈매1교** (대전청)
천안시 동남구 풍세면 삼태리 416
- ☐ **갈매2교** (대전청)
아산시 배방읍 갈매리 232-1
- ☐ **갈매3교** (대전청)
아산시 배방읍 갈매리 457
- ☐ **강경대교** (대전청)
논산시 채운면 삼거리 49-13
- ☐ **고명육교** (대전청)
제천시 고명동 727-6
- ☐ **고산교(상)** (대전청)
제천시 통안읍 장평리 1051-1
- ☐ **고산교(하)** (대전청)
제천시 통안읍 장평리 1051-1
- ☐ **과선교** (대전청)
서천군 마서면 송내리 274-3



bridge_database

❖ DB에 구성된 교량을 선택 및 검색을 통해 선택 → “선택 완료” 클릭 시 다음 화면으로

프로그램 사용 방법(2/3)

두 번째 화면 : 옵션 선택

와따가다리

선택 교량 미리보기: 17개 마커 표시



옵션 설정

1. 시작 날짜
2. 출장 가능 일수
3. Day별 근무(일괄 가능)
4. 점검시간(일괄 가능)
5. 캐시 사용

기능적 요구사항

- ✓ 카카오 API 사용
- ✓ 최적 경로화 알고리즘 구현 (단, 하루 근무시간 8시간제한)
- ✓ 점검 교량 위치 지도 표시
- ✓ 출발 위치 지정

선택 교량 위치 표시

옵션 설정

점검대상 다시 선택

서울특별시 동작구 신대방동 보라매로5가길 24

일정 최적화 실행 새로고침

시작 날짜(Day1): 2026-01-27

※ Day별 출발 시각/근무시간은 아래에서 설정합니다.

출장 가능 일수(N): 7

※ Day1~DayN만 생성/계산합니다. (최대 30일)

Day별 근무 시간(시작/끝)

	시작	끝
Day 1	08:00	16:00
Day 2	08:00	16:00
Day 3	08:00	16:00
Day 4	08:00	16:00
Day 5	08:00	16:00
Day 6	08:00	16:00
Day 7	08:00	16:00

· 30분 단위로 선택합니다.

· 끝 시간이 시작 시간보다 이르면 현재 화면에서는 적용하지 않습니다.

☐ 일괄적용 (점검시간을 전체 교량에 동일하게 적용)

분: 60

※ 체크 시 업로드/D8에 있는 점검시간을 덮어씁니다.

교량별 점검시간(분) - 일괄적용 OFF

갈매1교 (대전청)

60

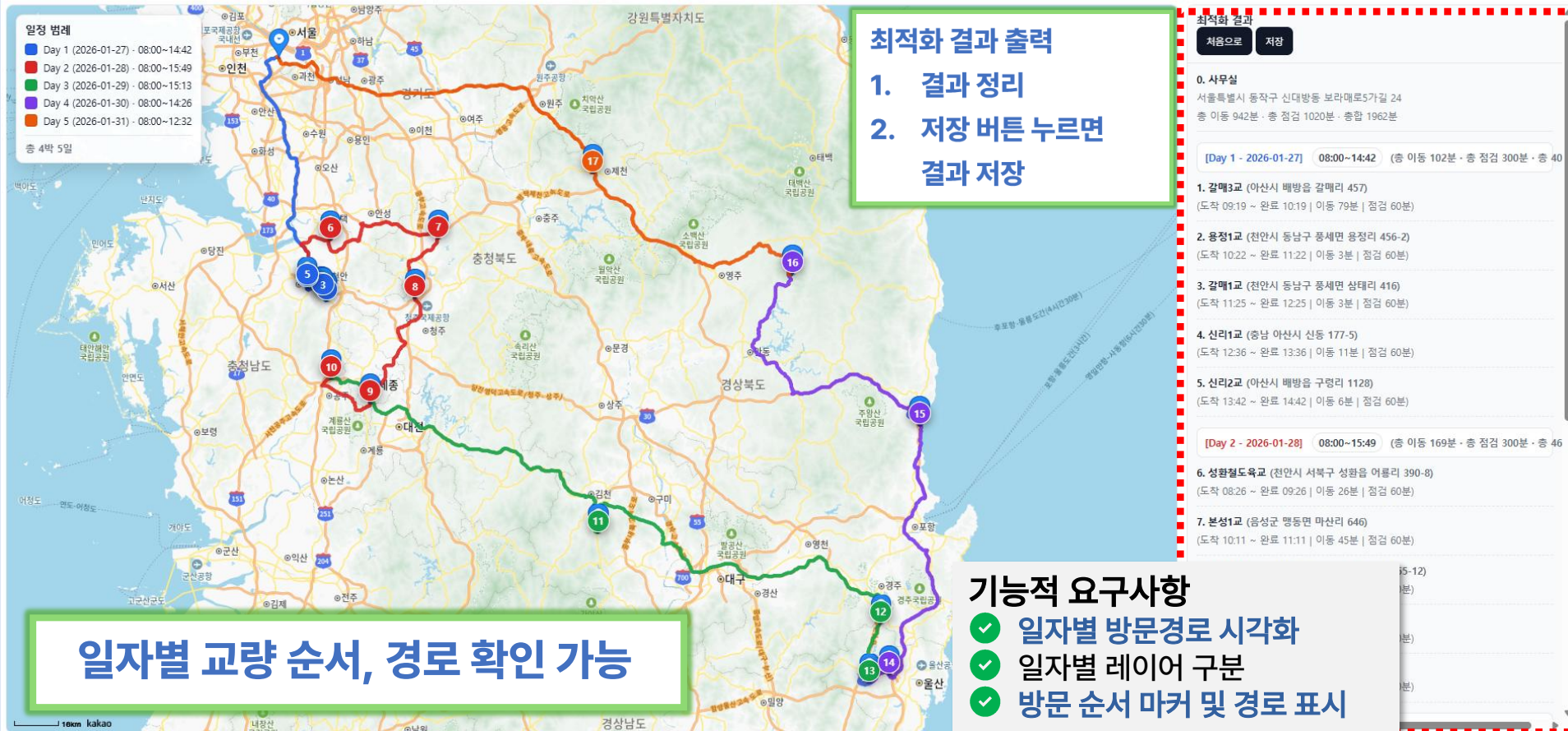
❖ 사용자에게 맞는 설정을 입력 후, " 일정 최적화 실행" 클릭 시 일정 계산

프로그램 사용 방법(3/3)

세 번째 화면 : 결과 출력 (지도 시각화 및 결과 정리)

와따가다리

최적화 완료(경로 렌더링 완료)



❖ 만약 다시 경로를 찾고 싶다면, "처음으로" 클릭

프로그램 사용 방법(3/3)

세 번째 화면 : 결과 출력 (지도 시각화 및 결과 정리)

와따가다리

최적화 완료(경로 렌더링 완료)

일정 범위

- Day 1 (2026-01-27) : 08:00~14:42
- Day 2 (2026-01-28) : 08:00~15:49
- Day 3 (2026-01-29) : 08:00~15:13
- Day 4 (2026-01-30) : 08:00~14:26
- Day 5 (2026-01-31) : 08:00~12:32

총 4박 5일

최적화 결과 출력

- 결과 정리
- 저장 버튼 누르면 결과 저장

최적화 결과

처음으로 저장

0. 사무실
서울특별시 동작구 신대방동 보라매로5가길 24
총 이동 942분 · 총 점검 1020분 · 총합 1962분

[Day 1 - 2026-01-27] 08:00~14:42 (총 이동 102분 · 총 점검 300분 · 총 40)

1. 갈매3교 (아산시 해방읍 갈매리 457)
(도착 08:18 ~ 완료 10:18 | 이동 70분 | 점검 60분)

용정리 456-2)
분 | 점검 60분)

상대리 416)
분 | 점검 60분)

5)
분 | 점검 60분)

1128)
분 | 점검 60분)

-15:49 (총 이동 169분 · 총 점검 300분 · 총 46)

6. 성환철도육교 (전주시 서북구 성환읍 어울리 390-8)
(도착 08:26 ~ 완료 09:26 | 이동 26분 | 점검 60분)

7. 본성1교 (음성군 맹동면 마산리 646)
(도착 10:11 ~ 완료 11:11 | 이동 45분 | 점검 60분)

8. 복현교(상) (정주시 정원구 오강읍 복현리 455-12)
(도착 11:39 ~ 완료 12:39 | 이동 28분 | 점검 60분)

9. 용수4교 (금남면 국곡리 130-6)
(도착 13:24 ~ 완료 14:24 | 이동 45분 | 점검 60분)

10. 목전IC교 (공주시 우성면 목천리 63-35)
(도착 14:49 ~ 완료 15:49 | 이동 25분 | 점검 60분)

일자별 교량 순서, 경로 확인 가능

❖ 만약 다시 경로를 찾고 싶다면, "처음으로" 클릭

최적 경로 탐색 방법(1/12)

최적 경로란 무엇인가?

최적 경로 = 정해진 시간 안에 가능한 많은 교량을
점검하고 가장 빨리 복귀하는 경로
(이동시간이 적을수록 근무자의 피로 ↓)

최적경로 우선순위 규칙

1. 교량 점검 최대
2. 출장일 최소
3. 점검 종료시간
(복귀 시간 포함)
4. 이동시간 최소

상위 순위가
동일하면 하위
순위로 결정

Ex) 최적 경로

경로 1

1. 교량 점검 = 8개
2. 출장일 = 3일
3. 점검 종료 시간 = 16시 2분
4. 이동시간 = 110분

=
=
<

1. 교량 점검 = 8개
2. 출장일 = 3일
3. 점검 종료 시간 = 16시 12분
4. 이동시간 = 120분

경로 2

출장 일정 = 방문 순서 결정 + 일정 분할 $\xrightarrow{\text{규칙 적용}}$ 최적 경로 결정

최적 경로 탐색 방법(2/12)

방문 순서 결정 : 전수 탐색 방법의 한계

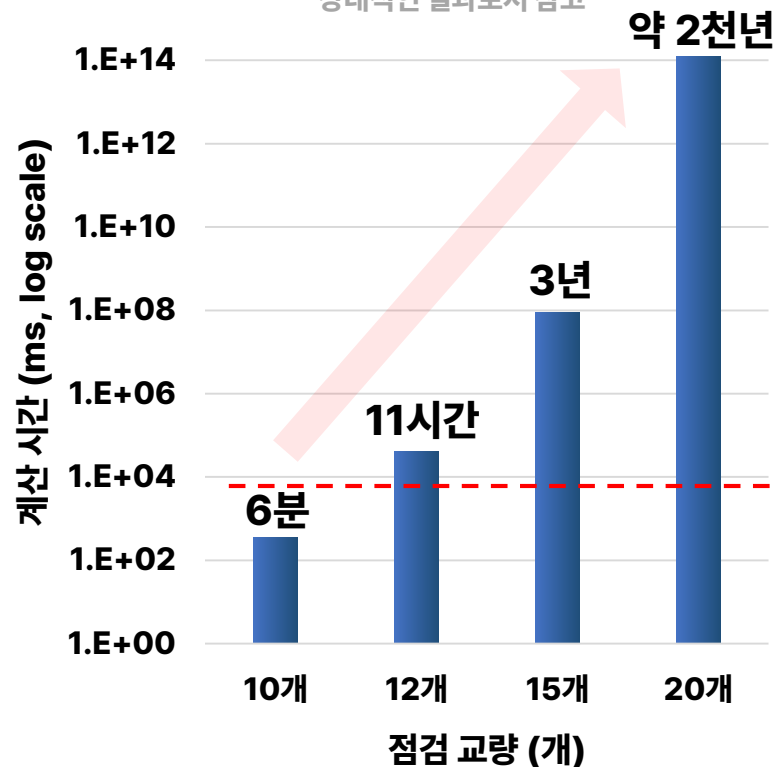
방문 순서 구하기 위한 가장 **간단, 정확한 방법**(전수 탐색) **모든 경로**의 수 중 **최적의 경로**를 선택하는 것한계점 : **계산량 폭증 (NP-Hard)**

교량 지점이 늘어날 수록 계산량 폭등

→ 전수 탐색 방법 불가능

교량 지점 별 계산시간

* 상대적인 결과로서 참고

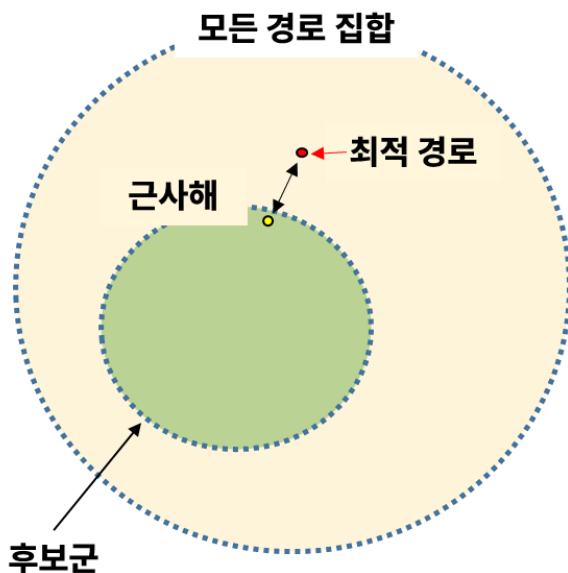


최적 경로 탐색 방법(3/12)

방문 순서 결정 : 효율적인 후보군 결정

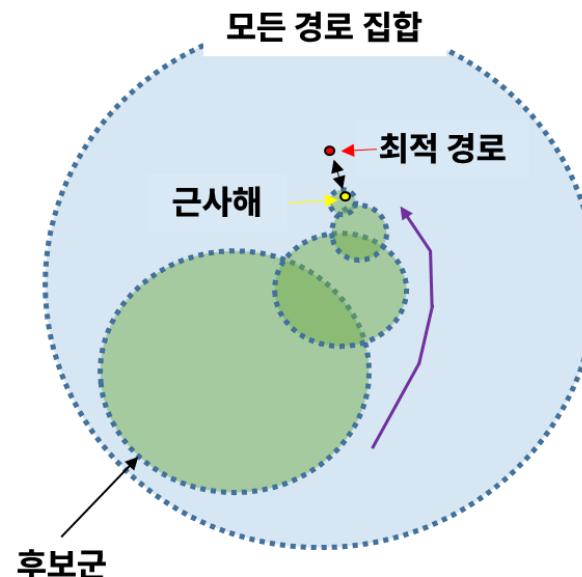
“최적경로와 가깝게 후보군을 형성하는 것이 매우 중요”

지역 탐색(간선 교환 방법, 2opt)



후보군에서 최적경로와 근접한 근사해 도출
(초기 부분집합이 중요)

광역 탐색(유전 알고리즘, GA)

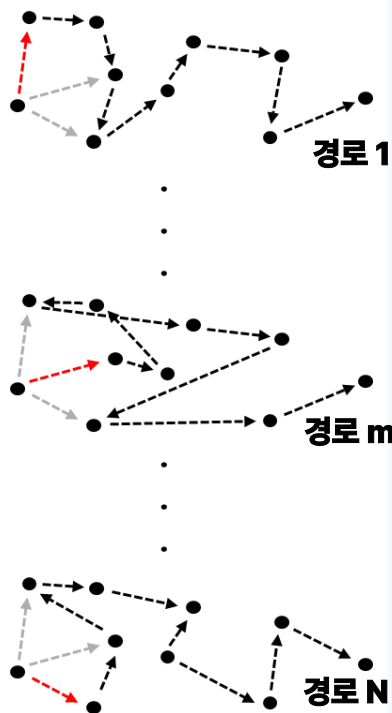


후보군이 반복에 따라 최적 경로에 가까워지며, 근사
해 도출 (얼마나 빨리 접근하는지가 중요)

최적 경로 탐색 방법(4/12)

방문 순서 결정 : 방법론의 구체적 메커니즘

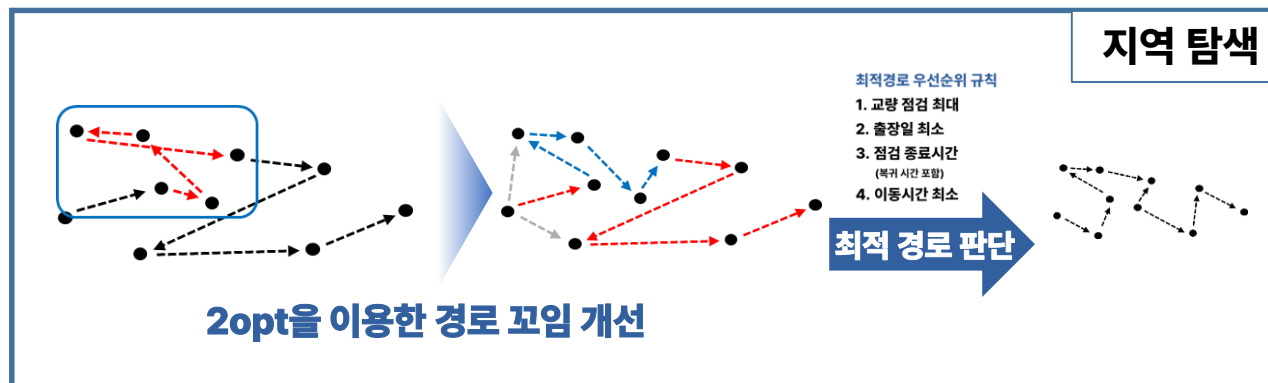
경로 후보군 생성



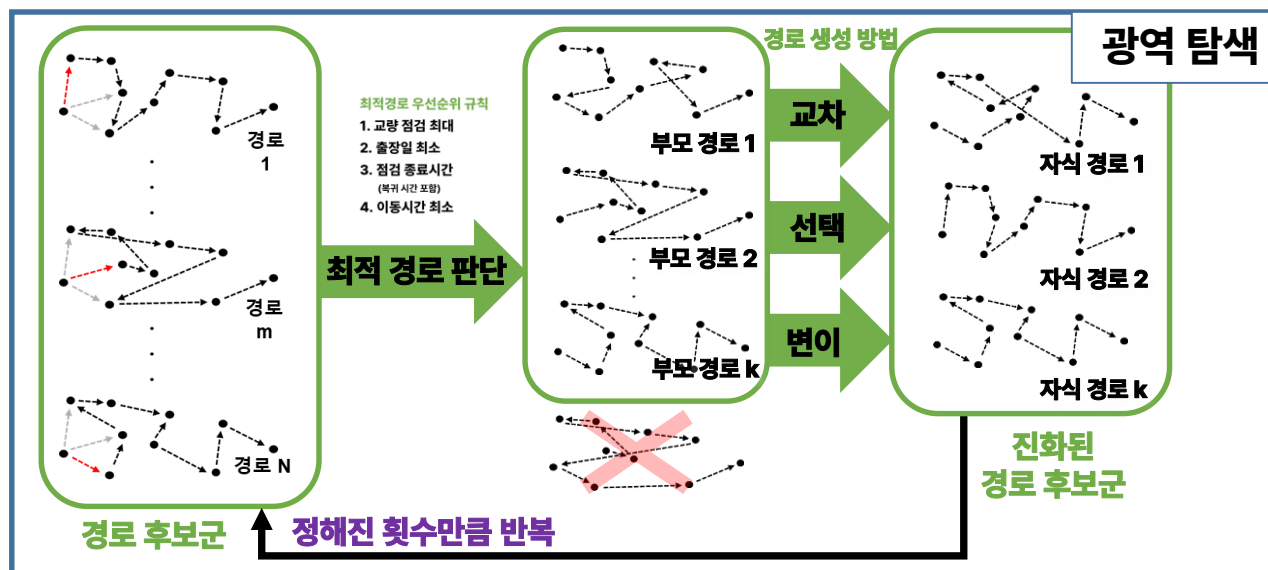
(k = 3, 총 N개 경로 생성)

Randomized NN
(가까운 k개 경로 랜덤 진출)

지역 탐색



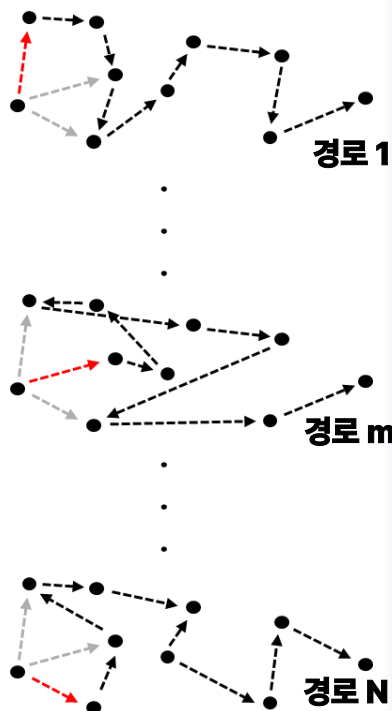
광역 탐색



최적 경로 탐색 방법(4/12)

방문 순서 결정 : 방법론의 구체적 메커니즘

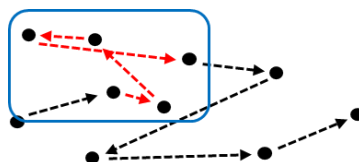
경로 후보군 생성



(k = 3, 총 N개 경로 생성)

Randomized NN
(가까운 k개 경로 랜덤 진출)

지역 탐색

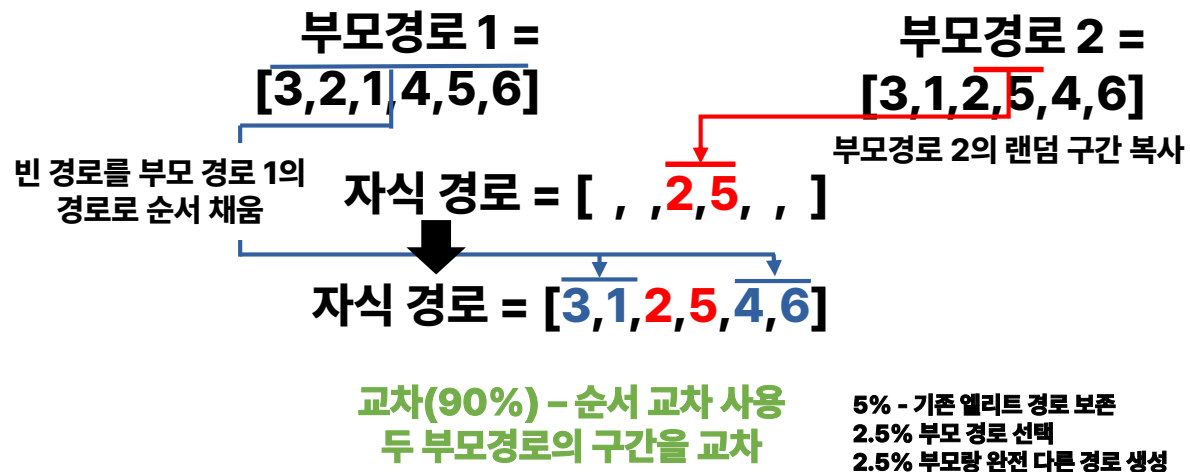


2opt을 이용한 경로 꼬임 개선

최적경로 우선순위 규칙

1. 교량 점검 최대
2. 출장일 최소
3. 점검 종료시간 (복귀 시간 포함)
4. 이동시간 최소

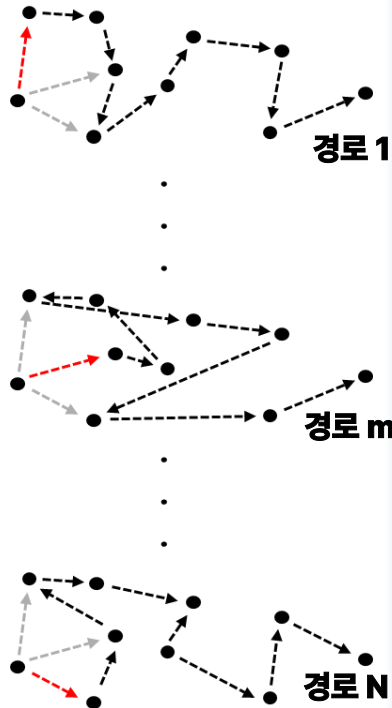
최적 경로 판단



최적 경로 탐색 방법(4/12)

방문 순서 결정 : 방법론의 구체적 메커니즘

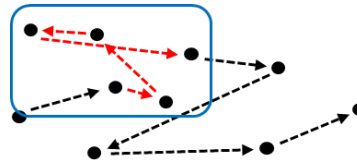
경로 후보군 생성



(k = 3, 총 N개 경로 생성)

Randomized NN
(가까운 k개 경로 랜덤 진출)

지역 탐색



2opt을 이용한 경로 꼬임 개선

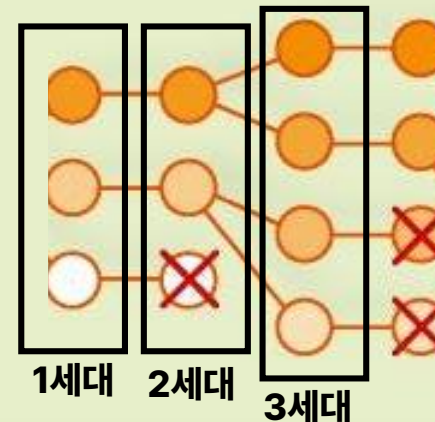
최적경로 우선순위 규칙

1. 교량 점검 최대
2. 출장일 최소
3. 점검 종료시간
(복귀 시간 포함)
4. 이동시간 최소

최적 경로 판단



빈 경로를 부모 경로로 순서



경로 2 =
2, 5, 4, 6]

의 랜덤 구간 복사

트 엘리트 경로 보존
경로 선택
완전 다른 경로 생성

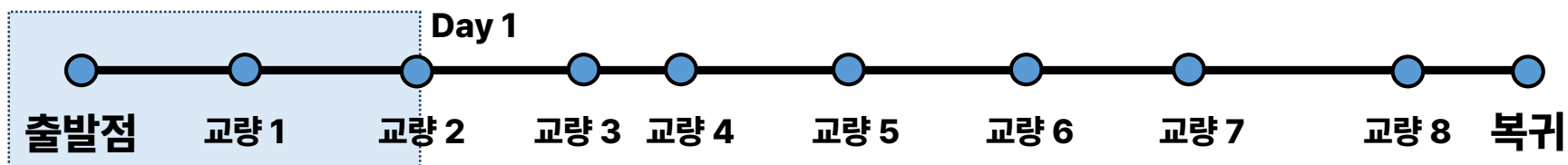
최적 경로 탐색 방법(5/12)

일정 분할 : 제약 조건 적용(하루 8시간)

“방문 순서 최적화로 결정된 전체 경로를 최적 분할이 필요”

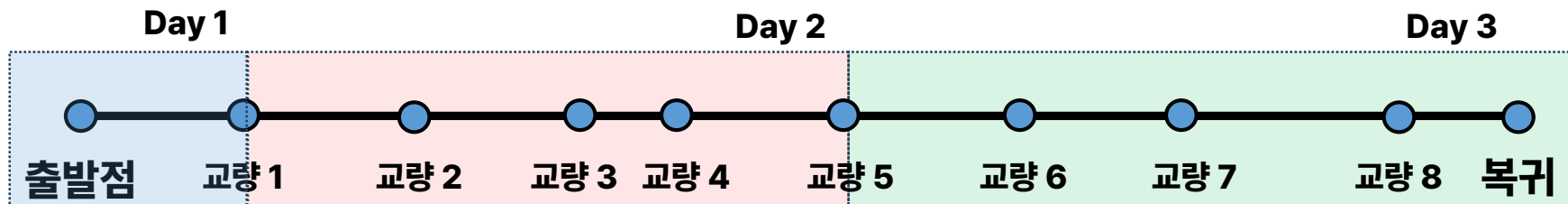
Static Split(정적 분할, SS) : 순차적 분할

교량 3을 Day 1에 넣으면 하루 근무시간 초과 → 다음날로 넘김 (전체 일정 고려 X)



Dynamic Split(동적 분할, DS) : 전구간 최적 분할 Dynamic Programming 알고리즘 이용

Day 1을 적게 하더라도 전체 일정이 빨리 끝나게 되면 해당 일정 분할 채택



최적 경로 탐색 방법(6/12)

방법론 성능 비교 실험(방문 순서 + 일정 분할 방식 조합)

실험 조건

1. 하루 근무시간 8시간
2. 전날 마지막 점검 교량 = 다음날 시작 지점
3. 처음 출발, 마지막 복귀
4. 점검시간은 60분 고정

최적 경로 평가 규칙

최적 경로 중요도 (4순위)

1. 미배정 교량
2. Day 수
3. 마지막 Day의 총 시간
4. 총 이동시간

*최적 경로 중요도 = 목적함수

방법론 구성

1. 전수 탐색 : 모든 경로의 경우의 수를 고려한 방법론 (검증용)

* 교량 10개이므로 오래 걸리더라도 결과 확인 가능

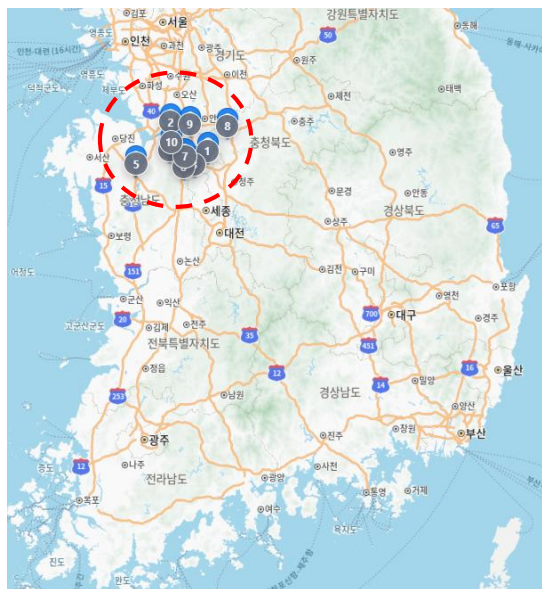
2. 3가지 방법론

검증 방법론	1번 방법론	2번 방법론	3번 방법론
전수 탐색 + 동적 분할	지역 탐색 + 동적 분할 (2opt + DS)	광역 탐색 + 정적 분할 (GA + SS)	광역 탐색 + 동적 분할 (GA + DS)

최적 경로 탐색 방법(7/12)

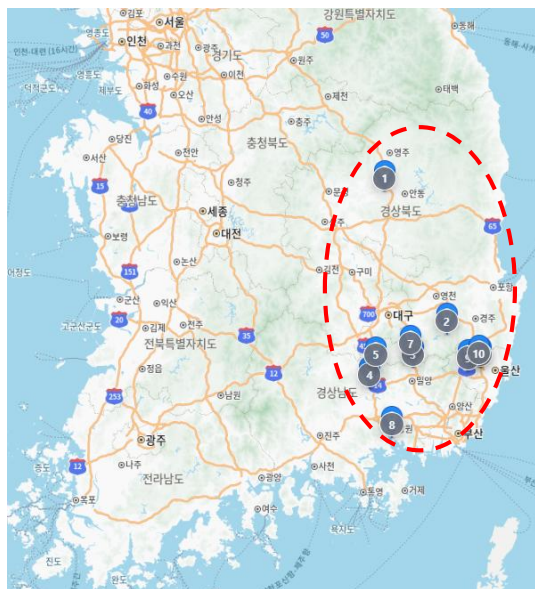
방법론 성능 비교 실험(방문 순서 + 일정 분할 방식 조합)

테스트 데이터 구성 (부산청 관할 교량, 대전청 관할 교량)



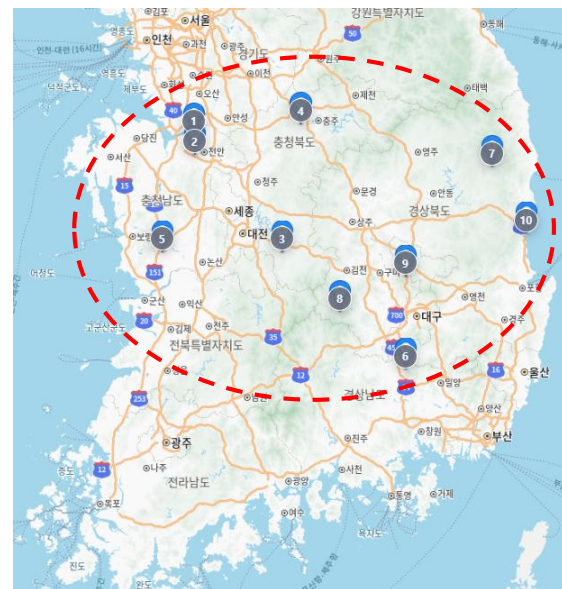
Test1_대전

병천교, 둔포램프2교, 조천4교, 금곡천교, 역탑1교, 차령교, 가송2교, 사곡IC교, 북천안IC5교, 양치교



Test2_부산

상리교(상), 제2신원교, 유천교, 화영교, 우천교, 구영육교, 청도2교, 귀산1교, 대천교, 범서IC1교



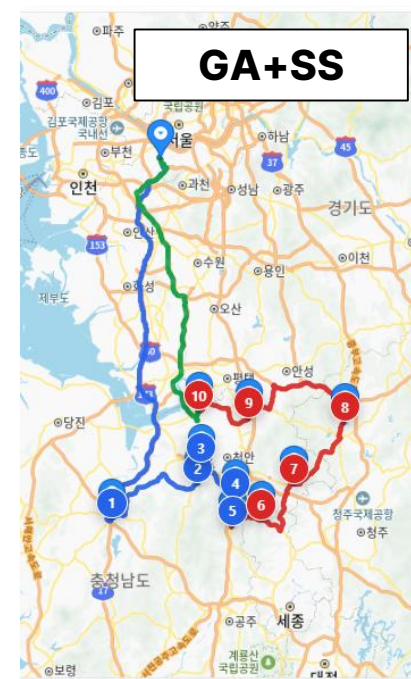
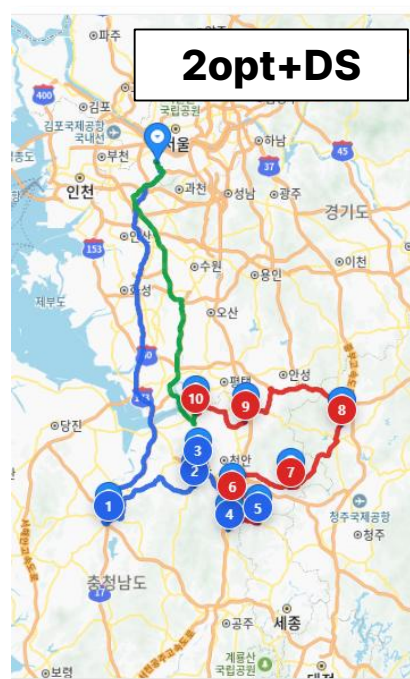
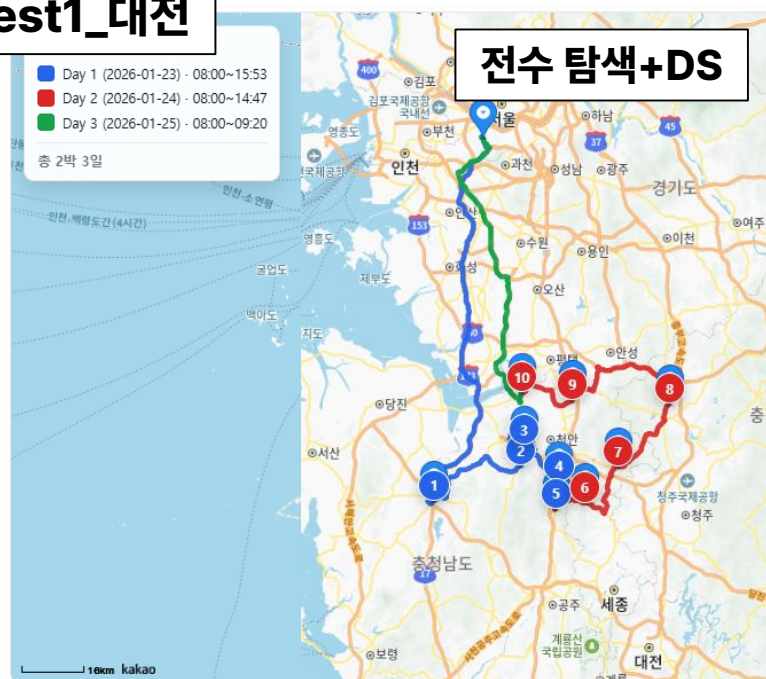
Test3_복합

둔포램프2교, 양치교, 과선교, 주덕교, 구룡교, 우천교, 섬촌교, 은적1교, 하판2교, 남호교

최적 경로 탐색 방법(8/12)

방법론 성능 비교 실험(방문 순서 + 일정 분할 방식 조합)

Test1_대전



*배정 교량 개수, DAY 수는 동물이므로 3순위인 복귀 시간으로 성능 평가

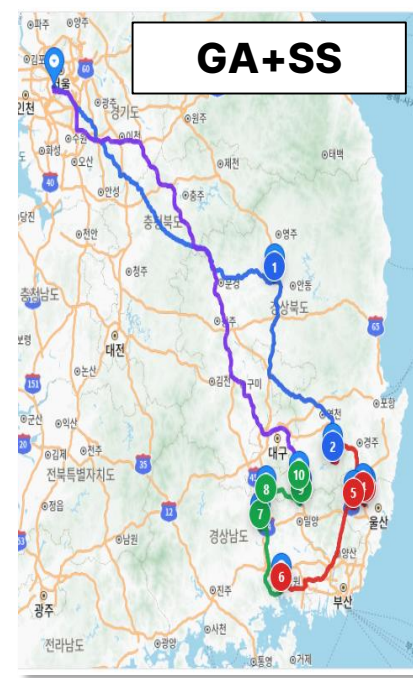
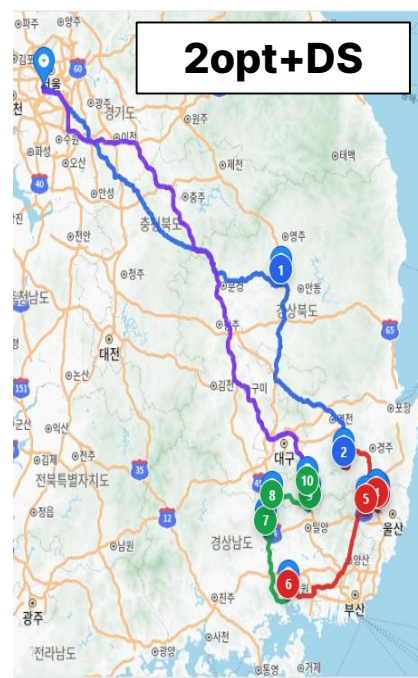
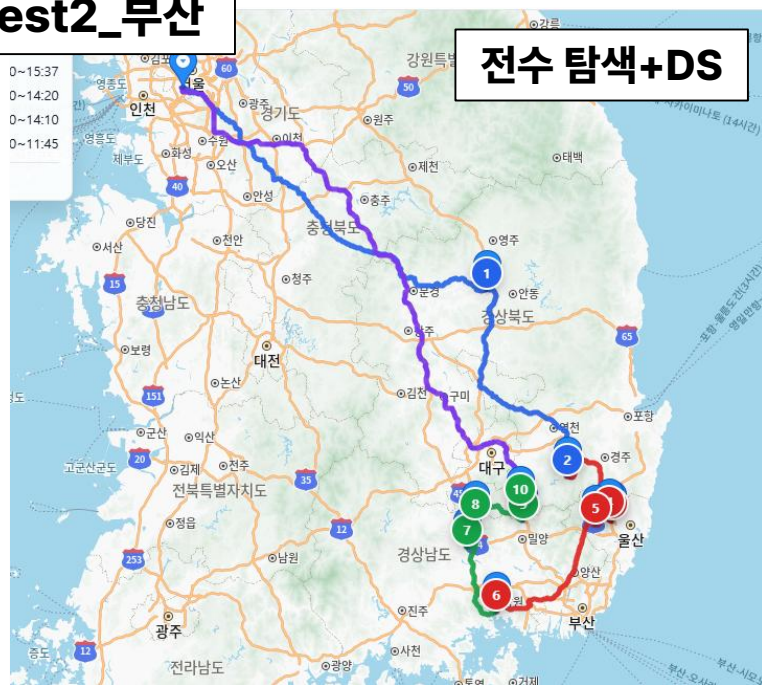
방법론	전수 탐색 + DS	① 2opt + DS	② GA + SS
Test1_대전	DAY2 16:07분 도착	DAY2 16:07분 도착	DAY2 16:10분 도착

* ㉓ GA + DS 는 검증용 데이터와 동일하게 나옴

최적 경로 탐색 방법(9/12)

방법론 성능 비교 실험(방문 순서 + 일정 분할 방식 조합)

Test2_부산



*배정 교량 개수, DAY 수는 동물이므로 3순위인 복귀 시간으로 성능 평가

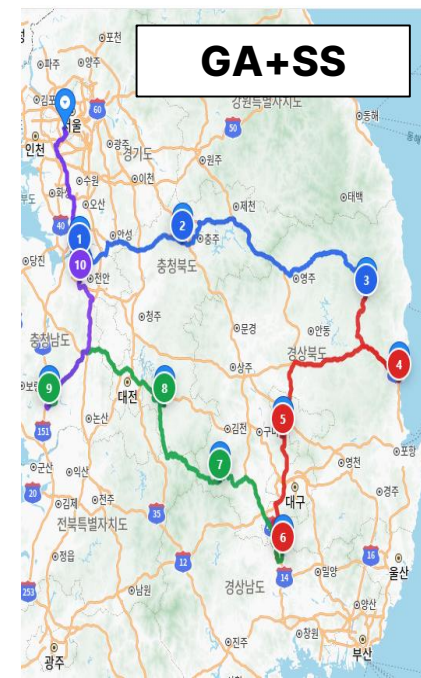
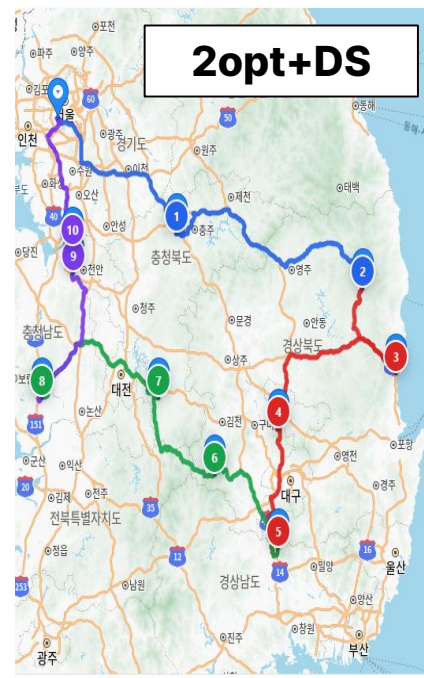
방법론	전수 탐색 + DS	① 2opt + DS	② GA + SS
Test2_부산	DAY3 17:45분 도착	DAY3 17:45분 도착	DAY3 17:45분 도착

* ㉓ GA + DS 는 검증용 데이터와 동일하게 나옴

최적 경로 탐색 방법(10/12)

방법론 성능 비교 실험(방문 순서 + 일정 분할 방식 조합)

Test3_복합



*배정 교량 개수, DAY 수는 동물이므로 3순위인 복귀 시간으로 성능 평가

방법론	전수 탐색 + DS	① 2opt + DS	② GA + SS
Test3_복합	DAY4 10:36분 도착	DAY4 12:37분 도착 + 2hours	DAY3 11:25분 도착 + 51mins

*③ GA + DS 는 검증용 데이터와 동일하게 나옴

최적 경로 탐색 방법(11/12)

방법론 성능 비교 실험(방문 순서 + 일정 분할 방식 조합)

성능 결과

방법론	전수 탐색 + DS	① 2opt + DS	② GA + SS	③ GA + DS
TEST1_대전	DAY2 16:07	DAY2 16:07	DAY2 16:10 (+ 3min)	DAY2 16:07
TEST2_부산	DAY3 17:45	DAY3 17:45	DAY3 17:45	DAY3 17:45
Test3_복합	DAY4 10:36	DAY4 12:37 (+ 2hours)	DAY4 11:25 (+ 51min)	DAY4 10:36

계산 소요 시간(api 호출 제외)

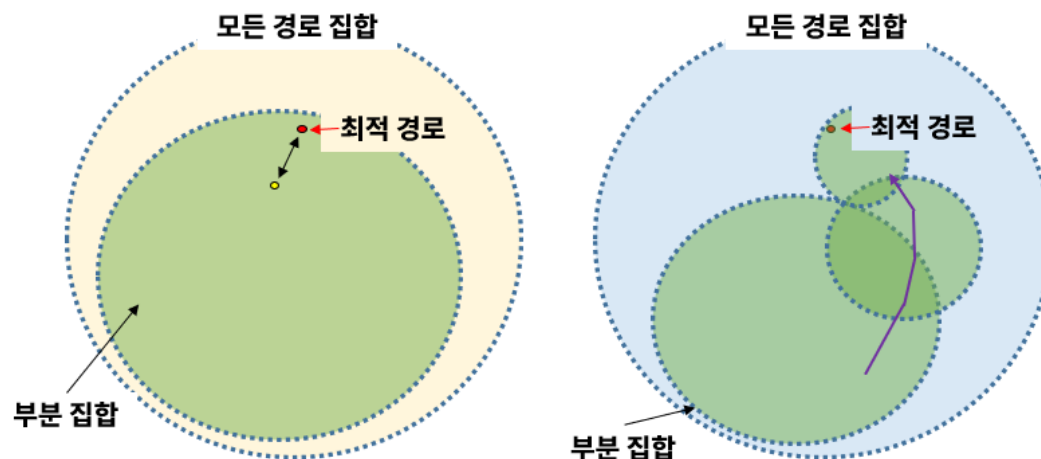
*길찾기 api 호출 시, 교량 1개 당 0.2~1초 소요

자원	전수 탐색 + DS	① 2opt + DS	② GA + SS	③ GA + DS
개인 노트북	56분 28초	1분 20초	2분 23초	15분
회사 컴퓨터	40분 22초	38초	1분 35초	4분 34초

최적 경로 탐색 방법(12/12)

방법론 성능 비교 실험 결론

부분 집합을 고려했는데, 최적 경로와 같은 이유



교량 10개의 전체 경로 집합은
생성되는 부분집합의 규모에 대해
포함 확률이 높았기 때문

최종 채택 방법론

적절한 성능에서 결과 도출 속도를 고려하여 데이터 상황에 맞게 자동으로 1,2 방법론 적용
단일 관할 데이터일 때는 **1번 방법론 적용**
복합 관할 데이터 일 때는 **2번 방법론 적용**

향후 발전 계획

발전 방향성 : 좀 더 편리하고 자동화 범위를 확대

기능 고도화 작업

- 신속 버전, 정밀 버전 으로 사용자가 선택할 수 있도록 만들기
*신속 버전(1,2번 방법론 통합) , 정밀 버전(3번 방법론(GA + DS))
- 결과 화면에서 Day 별 마지막 점검 교량 주변 숙소 위치 표시

분배 최적화 추가

- 점검 해야하는 전체 교량에 대해서 참여 팀, 팀별 출장 가능 일수를 입력하면 팀별로 점검 교량 분배하는 완전 자동화 출장 일정 수립 프로그램

프로젝트를 준비하면서 느끼고 배운 점

코드도 다 짜주는 AI 시대에 **나의 필요성과 생존 전략**에 대해서 고민할 수 있는 시간이 있습니다.

또한 프로젝트의 요구사항을 정확하게 이해하고 문제를 해결해 나가며 결과물을 만들어내는 **귀중한 경험**을 한 것 같습니다.

좋은 경험을 주셔서 감사합니다.

참고문헌

1. Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791–812.
2. Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
3. Oliver, I. M., Smith, D. J., & Holland, J. R. (1987). A study of permutation crossover operators on the traveling salesman problem. In *Proceedings of the Second International Conference on Genetic Algorithms* (pp. 224–230). Lawrence Erlbaum Associates.
4. Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12), 1985–2002.
5. Toth, P., & Vigo, D. (Eds.). (2014). *Vehicle Routing: Problems, Methods, and Applications* (2nd ed.). SIAM.

YOOSHIN

Q&A